Software Requirements Specification

For

Smart Bicycle Rental System (EcoRide)

Reg_no: _____

Roll No: _____

Section: V Semester / AI & DS

Prepared By: Sachin Godara

DATE: September 2026

Submitted To: Mr. Rohit Kumar

Department of Artificial Intelligence & Data Science

Network & System Programming (NSP) Minor Project

Executive Summary & Project Innovations

The EcoRide Smart Bicycle Rental System is an innovative, 3-tier desktop mobility platform developed in Java Swing and MySQL. It addresses critical last-mile connectivity and congestion issues on university and urban campuses while introducing sustainable micro-mobility incentives. In accordance with the academic requirements of the NSP Minor Project examination, this project introduces two key Unique Selling Propositions (USPs) that differentiate it from conventional bicycle rental platforms:

USP #1 – Dynamic Dock Rebalancing & Capacity Alerts: The system proactively monitors station dock utilization in real-time. When a station's available docks drop below 15%, it is flagged as STARVED; when capacity reaches ≥90%, it triggers a CONGESTED alert. If a rider tries to return a bicycle to a full station, the system blocks the docking and automatically recommends the nearest alternative station with estimated walking distance. This eliminates the common urban bike-sharing frustration of arriving at a full dock with no place to park.

USP #2 – Eco-Credit Rewards & CO_2 Offset Tracker: Every completed trip generates tangible environmental impact metrics. The system calculates CO_2 emissions avoided (0.21 kg/km), calories burned (7.5 kcal/min), and estimated trip distance (duration × 14 km/h average speed). For every 0.5 kg of CO_2 saved, the rider earns 1 Eco-Credit (equivalent to INR 1.00 wallet discount). This gamification transforms cycling from a simple transportation mode into a measurable sustainability initiative.

Table of Contents

1. Introduction
- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Product Scope
- 1.5 References

2. Overall Description
- 2.1 Product Perspective & 3-Tier Architecture
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation & Assumptions

3. External Interface Requirements
- 3.1 User Interfaces (Java Swing GUI Design)
- 3.2 Hardware Interfaces
- 3.3 Software Interfaces
- 3.4 Communications Interfaces

4. System Features
- 4.1 Feature 1: User Registration, Authentication & Wallet
- 4.2 Feature 2: Multi-Station Fleet Catalog & Availability
- 4.3 Feature 3: Smart Reservation & OTP Digital Unlock
- 4.4 Feature 4: Live Trip Metering & Active Concurrency HUD
- 4.5 Feature 5 [CORE USP #1]: Dynamic Dock Rebalancing
- 4.6 Feature 6 [CORE USP #2]: Eco-Credit Rewards & CO_2 Tracker
- 4.7 Feature 7: Docking Return, Automated Billing & Invoice
- 4.8 Feature 8: Maintenance & Fault Reporting System
- 4.9 Feature 9: Administrative Fleet Control & Analytics

5. Other Nonfunctional Requirements
- 5.1 Performance Requirements
- 5.2 Safety & Data Integrity
- 5.3 Security Requirements
- 5.4 Software Quality Attributes
- 5.5 Business Rules

6. Database Architecture & Relational Schema

Appendix A: Glossary of Technical Terms

Appendix B: Analysis Models (Use Case & Flowchart)

Appendix C: To Be Determined (TBD) List

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a rigorous, exhaustive description of the requirements and technical specifications for the Smart Bicycle Rental System (EcoRide) desktop client application. It establishes the foundational contract between system engineers, project evaluators, and academic faculty, adhering strictly to the IEEE Std 830-1998 Recommended Practice for Software Requirements Specifications. This document delineates both the functional requirements (user registration, digital unlock, concurrency metering, dynamic dock rebalancing, carbon offset tracking) and the non-functional attributes (sub-second performance, ACID data integrity, cryptographic security, and multi-tier architectural standards).

1.2 Document Conventions

This document follows the standardized academic formatting requirements mandated for the NSP Minor Project examination:

- Typography Standards: Body text is formatted in 12 pt Times New Roman with 1.15 line spacing; primary section headings are rendered in 14 pt bold; subheadings in 12 pt bold.
- Margin & Gutter Alignment: A dedicated left gutter margin of 1.25 inches is enforced on all body pages to accommodate clean, permanent spiral binding per the hard-copy submission requirements.
- Requirement Identifiers: Every individual functional requirement is tagged with a unique alphanumeric identifier in the format REQ-X.Y (e.g., REQ-5.1) to enable direct traceability across design, source code, and test cases.
- Priority Ratings: System features are formally prioritized into High (Core Baseline for Mid-Term-I), Medium (Operational Extension), and Low (Planned Post-Graduate Architecture).

1.3 Intended Audience and Reading Suggestions

This specification is engineered to provide actionable value across multiple stakeholder perspectives:

- Academic Examiners & Faculty Evaluators: Review Section 2.2 (Product Functions), Section 4.5 & 4.6 (Core USPs), and Appendix B (Analysis Models) to assess scope, academic rigor, and syllabus adherence.
- Software Engineers & Developers: Focus on Section 3 (External Interfaces), Section 4 (System Features), Section 6 (Database Schema), and concurrency notes.
- Quality Assurance & Verification Testers: Reference Section 4 functional acceptance criteria and Section 5 non-functional benchmarks to formulate

automated test suites.

1.4 Product Scope

The EcoRide Smart Bicycle Rental System is an autonomous 3-tier desktop platform engineered to transform urban and campus micro-mobility. Traditional bike-sharing networks suffer from severe operational bottlenecks, including station dock saturation (where riders arrive at full stations with nowhere to lock their bicycles), uneven fleet distribution, lack of user incentives, and opaque maintenance logging. EcoRide directly overcomes these problems by providing:

- End-to-End Ride Lifecycle Management from account creation through automated billing.
- Real-Time Concurrency with live trip metering on independent threads, ensuring a responsive GUI.
- Two Novel USPs: Dynamic Dock Rebalancing intelligence and Eco-Credit gamification engine.
- Complete Administrative Toolkit for fleet CRUD, station provisioning, and financial reporting.

1.5 References

1. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.

2. Java™ Platform, Standard Edition 17 API Specification (Oracle Corporation).

3. MySQL 8.0 Reference Manual (Oracle Corporation).

4. Smart Mobility & Shared Micromobility Operational Guidelines (2026).

2. Overall Description

2.1 Product Perspective & 3-Tier Architecture

EcoRide is an independent, self-contained desktop information system. It adopts a modular 3-tier architecture:

Tier 1 (Presentation): Java Swing graphical user interface running on the local client machine.

Tier 2 (Application / Business Logic): Core Java services handling validation, fare metering, carbon offset calculations, and thread coordination.

Tier 3 (Data Storage): MySQL relational database management system accessed via JDBC using thread-safe connection pooling.

```
[Java Swing GUI] <---> [Business Services & DAOs] <---> [JDBC API] <---> [MySQL 8.0 DB]
```

2.2 Product Functions

- User Authentication: Login/Registration with role segregation (Customer, Operator, Admin).
- Dock & Bike Discovery: Station filtering, bike type selection (Standard, Geared, Electric), battery level inspection.
- Reservation & Smart Unlock: 6-digit cryptographic PIN token generation to initiate a ride.
- Ride Monitoring: Live running duration clock, real-time fare accumulator, emergency assistance.
- Dynamic Rebalancing (USP #1): Proactive station saturation alerts and nearby return dock recommendations.
- Eco-Credit Engine (USP #2): Tangible carbon offset and fitness tracking with in-app wallet discounts.
- Automated Check-in & Invoicing: Automatic wallet deduction, transaction receipt generation, and station inventory sync.
- Maintenance Lifecycle: Fault flagging by riders/staff and technician repair management.

2.3 User Classes and Characteristics

1. Customer / Rider: Students, faculty, or campus visitors with basic computer skills. Expects a straightforward, visually appealing GUI, fast booking, transparent pricing, and instant receipts.

2. Station Operator / Technician: Field staff responsible for bike rebalancing, battery charging, and repair. Requires access to dock saturation levels and maintenance log entries.

3. System Administrator: IT/Operations managers responsible for adding stations, creating bicycles, setting base/hourly fares, and auditing financial reports.

2.4 Operating Environment

• Client Hardware: Any modern x86_64 computer with ≥4 GB RAM and 500 MB free disk space.

• Client Software: Windows 10/11, Linux, or macOS with OpenJDK 17 or higher.

• Database Engine: MySQL Server 8.0 or higher running locally on port 3306.

• Display Resolution: Optimized for 1280 × 720 and 1920 × 1080.

2.5 Design and Implementation Constraints

• Swing Framework: Built strictly using pure Java standard library Swing/AWT components for cross-platform portability.

• Thread Safety: UI rendering is strictly confined to the Event Dispatch Thread (EDT). Long-running background operations utilize dedicated threads or javax.swing.Timer.

• Database Security: All SQL interactions must employ java.sql.PreparedStatement to prevent SQL Injection.

• Offline/Local Capability: The system must run autonomously on a standalone local machine without cloud connectivity.

2.6 User Documentation & Assumptions

Comprehensive user manual embedded in the application under Help → User Manual. Database initialization and configuration instructions provided in db/schema.sql and README.md.

Assumptions: The target machine has MySQL Server running with database bicycle_rental_db created and credentials configured. MySQL Connector/J JDBC driver library (mysql-connector-j-8.x.jar) is placed on the Java classpath.

3. External Interface Requirements

3.1 User Interfaces (Java Swing GUI Design)

The application user interface adheres to modern ergonomics, utilizing high-contrast color palettes, rounded borders, card layouts, and responsive font scaling:

• UI-1 (Login & Sign-Up): Dual tabbed/dialog interface for credential verification and new account registration with instant form validation cues.

• UI-2 (Customer Dashboard): Header displaying active user profile, wallet balance, and accumulated Eco-Credits; center panel containing metrics cards (Active Rides, Total KM, CO_2 Saved); navigation toolbar.

• UI-3 (Bike Catalog & Reservation): Split pane interface with station list on the left and available bicycle cards/table on the right with battery % meters.

• UI-4 (Active Ride HUD): Dedicated dialog featuring a running stopwatch (00:15:32), live estimated fare (₹15.00), and "End Ride & Dock" action button.

• UI-5 (Receipt & Billing Dialog): Formatted modal invoice display with print/save options.

• UI-6 (Admin / Fleet Console): Tabbed table views for bicycle CRUD, station capacity configuration, maintenance ticket resolution, and revenue charts.

3.2 Hardware Interfaces

• Standard Keyboard and Mouse for user input.

• Monitor resolution ≥ 1280 × 720.

• Optional local printer for physical invoice printing.

3.3 Software Interfaces

• Java Virtual Machine: OpenJDK 17.0+.

• JDBC Driver: MySQL Connector/J 8.0+.

• DBMS: MySQL Community Server 8.0+.

3.4 Communications Interfaces

TCP/IP socket connection over port 3306 for JDBC client-to-database communication.

4. System Features

4.1 Feature 1: User Registration, Authentication & Wallet

Description: Allows new users to create an account, log in securely, and manage their pre-paid wallet balance.

Priority: High (Core Baseline)

Functional Requirements:

- REQ-1.1: System shall validate that username and email are unique across all accounts.
- REQ-1.2: Password must be securely hashed using SHA-256 before storage; no plaintext passwords permitted.
- REQ-1.3: System shall initialize new customer accounts with a default introductory wallet balance (e.g., INR 200.00).
- REQ-1.4: Users shall have the ability to top up their wallet balance using simulated payment methods (UPI, Card, Cash).

4.2 Feature 2: Multi-Station Fleet Catalog & Availability

Description: Displays all registered docking stations and lists available bicycles filtered by type.

Priority: High (Core Baseline)

Functional Requirements:

- REQ-2.1: System shall retrieve and display all active stations with total docks and current available bike count.
- REQ-2.2: System shall allow filtering bicycles by category: Standard, Geared, and Electric.
- REQ-2.3: For Electric bicycles, the current battery percentage must be visibly indicated with a color-coded status bar.

4.3 Feature 3: Smart Reservation & OTP Digital Unlock

Description: Coordinates the bicycle checkout process and generates a secure one-time unlock code.

Priority: High (Core Baseline)

Functional Requirements:

- REQ-3.1: System shall verify that the user has a minimum wallet balance of at least INR 50.00 before reservation.

• REQ-3.2: System shall verify that the user does not have an existing unfinished active rental.

• REQ-3.3: Upon booking confirmation, the system shall generate a random 6-digit numeric unlock PIN.

• REQ-3.4: System shall atomically transition the bicycle status to IN_USE and decrement the station's available bike count.

4.4 Feature 4: Live Trip Metering & Active Concurrency HUD

Description: Tracks the ride in real time on an independent thread, metering duration and calculating accrued costs.

Priority: High (Core Baseline)

Functional Requirements:

• REQ-4.1: System shall run a background timer updating the elapsed time every second without freezing the Swing GUI.

• REQ-4.2: Base fare (e.g., INR 10.00) applies to the first 15 minutes; subsequent minutes are metered at a fixed per-minute rate according to the bike type.

• REQ-4.3: If the application is unexpectedly closed, the active rental status remains persisted in the database with original start timestamp.

4.5 Feature 5 [CORE USP #1]: Dynamic Dock Rebalancing & Capacity Alerts

Description: Proactively monitors station capacity to prevent dock saturation and guides riders to nearby stations.

Priority: High (Core Baseline)

Functional Requirements:

• REQ-5.1: When a station's available docks drop below 15%, the system flags it as STARVED and notifies operators.

• REQ-5.2: When a station reaches ≥90% capacity, the system flags it as CONGESTED.

• REQ-5.3: If a rider attempts to return a bicycle to a station with zero empty docks, the system blocks the docking and automatically displays the nearest alternative station with walking distance.

4.6 Feature 6 [CORE USP #2]: Eco-Credit Rewards & CO_2 Offset Tracker

Description: Computes environmental and health impact metrics, awarding redeemable loyalty credits.

Priority: High (Core Baseline)

Functional Requirements:

- REQ-6.1: System shall estimate trip distance: Distance (km) = Duration (hours) × 14 km/h (average speed).
- REQ-6.2: System shall compute CO_2 emissions avoided: CO_2 Saved (kg) = Distance (km) × 0.21 kg/km.
- REQ-6.3: System shall compute calories burned: Calories (kcal) = Duration (min) × 7.5 kcal/min.
- REQ-6.4: For every 0.5 kg of CO_2 saved, the user is awarded 1 Eco-Credit (1 Credit = INR 1.00 discount).

4.7 Feature 7: Docking Return, Automated Billing & Invoice

Description: Concludes the rental session, updates inventory, debits wallet balance, and issues an itemized receipt.

Priority: High (Core Baseline)

Functional Requirements:

- REQ-7.1: User selects destination station; system confirms empty dock availability.
- REQ-7.2: Total fare is calculated; Eco-Credits are applied if requested by the user.
- REQ-7.3: Net amount is automatically deducted from the user's wallet balance within a single transactional boundary.
- REQ-7.4: System generates an itemized invoice dialog displaying Rental ID, Duration, Fare, Eco-Discount, and Final Balance.

4.8 Feature 8: Maintenance & Fault Reporting System

Description: Enables riders or staff to report mechanical issues, automatically locking the bike from further checkout.

Priority: Medium (Operational Feature)

Functional Requirements:

- REQ-8.1: Rider can report issues (Flat Tire, Brakes, Chain, Battery) during return.
- REQ-8.2: Reported bikes are immediately assigned status MAINTENANCE.
- REQ-8.3: Technicians can view open maintenance tickets, enter repair costs, and transition status back to AVAILABLE.

4.9 Feature 9: Administrative Fleet Control & Analytics

Description: Full administrative dashboard for fleet management, station oversight, and revenue monitoring.

Priority: Medium (Operational Feature)

Functional Requirements:

- REQ-9.1: Admins can perform full CRUD operations on Bicycles and Stations.
- REQ-9.2: System provides summary metrics: Total Fleet Count, Active Rentals, Total Revenue Generated, and Total Campus CO_2 Offset.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Query execution and table reloading must complete within <150 ms.
- Memory footprint of the client application must remain under 150 MB RAM.

5.2 Safety & Data Integrity

In the event of system crash or sudden power termination, active rental states must remain intact in MySQL to prevent data corruption or lost rental records.

5.3 Security Requirements

- Passwords must be hashed using SHA-256 with salt.
- Role-based authorization: Customers cannot access administrative CRUD panels.
- Parameterized SQL queries must be enforced across all database queries.

5.4 Software Quality Attributes

- Maintainability: Clear separation of concerns following the MVC and DAO design patterns.
- Portability: Code runs without modification on Windows, macOS, and Linux systems with standard Java 17 runtime.
- Robustness: Comprehensive try-catch-finally blocks with meaningful error dialogs presented to the user instead of raw terminal stack traces.

5.5 Business Rules

- Minimum age requirement: 16 years.
- Maximum rental duration per single session: 24 hours (beyond which penalty rates apply).
- Minimum wallet balance to initiate a ride: ₹50.00.

6. Database Architecture & Relational Schema

The EcoRide system is backed by a normalized MySQL relational database (bicycle_rental_db) designed in Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. The schema comprises the following core tables:

6.1 Table: users

Stores all registered system accounts with role-based access control.

Column	Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address
password_hash	VARCHAR(256)	NOT NULL	SHA-256 hashed password
role	ENUM	NOT NULL	CUSTOMER/OPERATOR/ADMIN
wallet_balance	DECIMAL(10,2)	DEFAULT 200.00	Pre-paid wallet amount

6.2 Table: stations

Represents physical docking locations across the campus/city network.

Column	Type	Constraints	Description
station_id	INT	PK, AUTO_INCREMENT	Unique station identifier
station_name	VARCHAR(100)	NOT NULL	Display name
location	VARCHAR(200)	NOT NULL	Physical address
total_docks	INT	NOT NULL	Total parking slots
available_bikes	INT	NOT NULL	Currently available bikes

6.3 Table: bicycles

Represents individual bicycles in the fleet with type classification.

Column	Type	Constraints	Description
bike_id	INT	PK, AUTO_INCREMENT	Unique bicycle identifier
station_id	INT	FK → stations	Home station reference
bike_type	ENUM	NOT NULL	STANDARD/GEARED/ELECTRIC
status	ENUM	NOT NULL	AVAILABLE/IN_USE/MAINTENANCE
battery_pct	INT	NULL (electric only)	Battery charge percentage
hourly_rate	DECIMAL(8,2)	NOT NULL	Per-hour rental cost

6.4 Table: rentals

Records every booking transaction from checkout to return.

Column	Type	Constraints	Description
rental_id	INT	PK, AUTO_INCREMENT	Unique rental identifier
user_id	INT	FK → users	Renting customer
bike_id	INT	FK → bicycles	Rented bicycle
pickup_station_id	INT	FK → stations	Start station
dropoff_station_id	INT	FK → stations (nullable)	End station
start_time	DATETIME	NOT NULL	Ride start timestamp
end_time	DATETIME	NULL	Ride end timestamp
total_fare	DECIMAL(10,2)	NULL	Computed fare amount
co2_saved_kg	DECIMAL(8,4)	NULL	Estimated CO_2 offset

Appendix A: Glossary of Technical Terms

Term	Definition
Dock	A physical or virtual parking slot at a station where a bicycle is secured.
Dock Saturation	The percentage of occupied docks at a given station.
Eco-Credit	A virtual loyalty point awarded to riders based on greenhouse gas emissions saved.
DAO	Data Access Object – a design pattern providing an abstract interface to a database.
EDT	Event Dispatch Thread – Java Swing's thread for dispatching GUI events.
JDBC	Java Database Connectivity – standard API for connecting Java to databases.
OTP	One-Time Password/PIN – a temporary authentication code.
CRUD	Create, Read, Update, Delete – the four basic operations of persistent storage.
SHA-256	Secure Hash Algorithm producing a 256-bit hash digest for password security.
3NF	Third Normal Form – a database normalization level eliminating transitive dependencies.

Appendix B: Analysis Models (Use Case & Flowchart)

B.1 High-Level Use Case Model

```
SMART BICYCLE RENTAL SYSTEM
==========================================

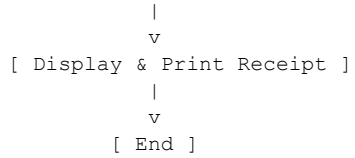
(Customer) --> [ Register & Login ]
           --> [ Browse Stations & Bikes ]
           --> [ Unlock Bike with OTP ]
           --> [ View Active Trip Timer ]
           --> [ Return Bike & Pay Fare ]
           --> [ Earn Eco-Credits & Track CO2 ]   <-- (USP)
           --> [ Report Maintenance Issue ]

(Operator) --> [ Monitor Dock Saturation ]        <-- (USP)
           --> [ Rebalance Station Fleet ]
           --> [ Service & Repair Bicycles ]

(Admin)    --> [ CRUD Bicycles & Stations ]
           --> [ Configure Rates & Docks ]
           --> [ View Revenue & Campus Green Reports ]
```

B.2 System Flowchart (Rental Lifecycle)

```
[ Start: User Login ]
        |
        v
[ Check Wallet Balance >= Rs.50 ] --(No)--> [ Prompt Wallet Top-up ]
        | (Yes)
        v
[ Select Station & Bicycle ]
        |
        v
[ Generate 6-Digit Unlock PIN ]
        |
        v
[ Update DB: Bike=IN_USE, Station Count - 1 ]
        |
        v
[ Start Active Trip Timer (EDT Safe) ]
        |
        v
[ Rider Reaches Destination ]
        |
        v
[ Check Destination Dock Capacity ]
    |                   |
  (Full)            (Available)
    |                   |
    v                   v
[ Suggest Nearest   [ Confirm Dock Return ]
  Alternative           |
  Station ]             v
             [ Calculate Duration, Fare, CO2 Saved ]
                        |
                        v
             [ Apply Eco-Credits Discount if Selected ]
                        |
                        v
             [ Debit User Wallet & Commit DB Txn ]
```


|
v
[Display & Print Receipt]
|
v
[End]

Appendix C: To Be Determined (TBD) List

TBD-1: Physical IoT hardware lock integration via MQTT protocol (planned for future post-graduate extension).

TBD-2: Integration with third-party payment gateway APIs (e.g., Razorpay/Stripe webhooks).

TBD-3: GPS-based real-time bicycle tracking on a campus map overlay.

TBD-4: Mobile companion application for Android/iOS platforms.